ISSN 2383-6318(Print) / ISSN 2383-6326(Online)
KIISE Transactions on Computing Practices, Vol. 32, No. 3, pp. 105-118, 2026. 3
https://doi.org/10.5626/KTCP.2026.32.3.105

엣지 컴퓨팅에서 분산 인공지능 워크로드를 위한 리소스 셀 구조

(Resource-Cell Structure for Distributed AI Workloads in Edge Computing)

권 용 훈 † 정 인 범 ††
(Yonghun Kwon) (Inbum Jung)

요 약 인공지능 서비스 수요 증가는 중앙 집중형 클라우드 서버의 에너지 소비와 지연 문제를 심화시키고 있다. 이를 해결하기 위해 엣지 컴퓨팅이 제안되었지만, 이기종 디바이스의 자원 제약으로 안정적인 서비스 제공에 한계가 있다. 본 논문은 이러한 문제를 해결하기 위해 디바이스 자원을 정량화하는 구조를 제안한다. 리소스 셀은 CPU 성능과 메모리 대역폭을 반영한 인공지능 워크로드 기반 벤치마크를 통해 산출되며, 엣지 디바이스 자원을 정량화한다. 본 연구에서는 엣지 디바이스인 Raspberry Pi를 대상으로 벤치마크를 수행하고, MQTT 기반 분산 추론 환경에서 작업 분배 전략을 적용하였다. 실험 결과, 리소스 셀 기반 추론 예측은 최대 94%의 높은 추론 시간 예측 신뢰도를 보였으며, 에너지 효율성과 분산 처리 효율성을 확인하였다. 본 연구는 이기종 엣지 환경에서 효율적 자원 관리와 분산 추론을 위한 구조적 접근을 제시한다.

키워드: 엣지컴퓨팅, 분산인공지능, 사물인터넷, 벤치마크, 엣지디바이스, 자원제약성

Abstract The increasing demand for artificial intelligence (AI) services intensifies energy consumption and latency issues in centralized cloud servers. Although edge computing addresses some challenges, limited resources in heterogeneous device environments restrict stable operation. This paper proposes Resource-Cell structure, a quantitative abstraction of device resources derived from AI workload-based benchmarking of CPU performance and memory bandwidth. In this study, benchmarking was performed on Raspberry Pi edge devices, and a task distribution strategy was applied in an MQTT-based distributed inference environment. The Resource-Cell-based inference prediction demonstrated an inference time prediction accuracy of up to 94%, while effectively validating both energy efficiency and distributed processing efficiency. This study introduces a structural methodology for efficient resource management and distributed inference in heterogeneous edge environments.

Keywords: edge computing, distributed artificial intelligence, internet of things, benchmark, edge device, resource constraint

· 이 논문은 2024년도 강원대학교 대학회계의 지원을 받아 수행한 연구임

† 학생회원 : 강원대학교 컴퓨터정보통신공학과 학생
russell1113@kangwon.ac.kr
†† 종신회원 : 강원대학교 컴퓨터공학과 교수(Kangwon Nat'l Univ.)
ibjung@kangwon.ac.kr
(Corresponding author임)

논문접수 : 2025년 8월 29일
(Received 29 August 2025)
논문수정 : 2026년 1월 28일
(Revised 28 January 2026)
심사완료 : 2026년 1월 29일
(Accepted 29 January 2026)

Copyright©2026 한국정보과학회 : 개인 목적이나 교육 목적인 경우, 이 저작물의 전체 또는 일부에 대한 복사본 혹은 디지털 사본의 제작을 허가합니다. 이 때, 사본은 상업적 수단으로 사용할 수 없으며 첫 페이지에 본 문구와 출처를 반드시 명시해야 합니다. 이 외의 목적으로 복제, 배포, 출판, 전송 등 모든 유형의 사용행위를 하는 경우에 대하여는 사전에 허가를 얻고 비용을 지불해야 합니다.
정보과학회 컴퓨팅의 실제 논문지 제32권 제3호(2026. 3)

1. 서 론

클라우드 컴퓨팅은 데이터 센터의 방대한 자원과 연산 능력을 바탕으로 ChatGPT와 같은 대규모 인공지능 서비스를 처리하는 핵심 기술로 자리 잡았다. 그러나 최근 인공지능 서비스 수요가 기하급수적으로 증가하면서, 데이터 센터의 확장 및 신규 설립이 지속적으로 추진되고 있다. 이는 NVIDIA, Google 등 주요 기업들이 개발한 인공지능 칩 기반의 대규모 데이터 센터에서의 서버 운영 수요 증가에 기인하며, 특히 생성형 인공지능 서비스의 급증으로 인해 막대한 에너지가 소비된다. 이러한 요구들은 발전소 및 냉각 인프라 증설에 따른 막대한 자원 소모로 이어진다[1]. 또한, 수많은 사용자 요청이 클라우드 서버에 집중됨에 따라 서버 과부하, 네트워크 병목으로 인한 지연(latency)이 발생하고, 이는 인공지능 서비스 품질 저하로 이어진다. 아울러 데이터 센터 기반의 클라우드 컴퓨팅은 중앙집중형 구조로 인해, 수집한 사용자 개인 정보 보안은 아직까지 해결해야 하는 과제로 남아있다[2].

클라우드 컴퓨팅의 이러한 문제점들을 해결하기 위하여 엣지 컴퓨팅(Edge Computing)이 제안되었다. 엣지 컴퓨팅은 데이터를 데이터 센터로 보내는 것이 아니라 사용자 근처에 위치한 디바이스에서 처리함으로써 클라우드 컴퓨팅에서의 시간 지연과 에너지 소모를 줄이는 패러다임이다[3]. 그러나 엣지 환경은 연산 성능과 자원 상태가 디바이스마다 상이한 이기종 환경이며, 각 디바이스의 성능 또한 제한적이다. 엣지 컴퓨팅 환경의 이기종성과 제한된 성능은 클러스터 구성 시 디바이스 선택에 중요한 제약으로 작용하며, 이는 인공지능 서비스의 품질에 큰 영향을 미친다[4].

그림 1은 엣지 컴퓨팅 환경에서 이기종성과 배터리 의존성이 동시에 존재하는 하나의 시나리오를 보여준다. 서로 다른 연산 성능과 배터리 상태를 가진 다수의 엣지 자원들이 하나의 엣지 서버에 연결된 환경을 나타낸다. 엣지 서버가 분산 처리를 위해 새로운 디바이스를 선택하는 과정에서, 디바이스는 단순한 연산 성능 비교만으로 평가될 수 없으며, 하드웨어 특성과 잔여 배터리 수준을 함께 고려해야 한다. 이처럼 연산 자원이 풍부한 디바이스와 배터리 상태가 우수한 디바이스 중 어떤 자원을 선택하는 것이 더 유리한지 판단하기 어려운 문제가 발생한다. 고성능 디바이스라 하더라도 배터리 잔량이 부족할 경우 서비스가 수행 도중 중단될 수 있으며, 반대로 배터리 상태는 양호하나 연산 성능이 낮은 디바이스를 선택할 경우 전체적인 처리 지연으로 이어질 수 있다.

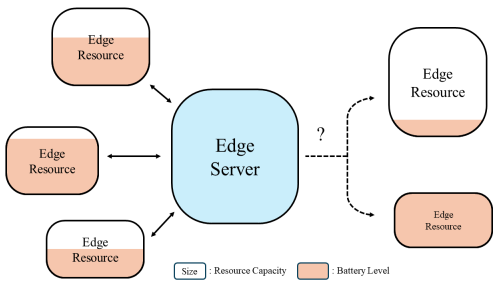

그림 1 엣지 네트워크 구성의 우선순위
Fig. 1 Priorities in Edge Network Configuration

본 연구에서는 이러한 문제를 해결하기 위해, 엣지 컴퓨팅에서 클러스터에 참여하는 디바이스들의 보유 자원을 정량화한 리소스 셀(Resource-Cell) 구조를 제안한다. 리소스 셀은 디바이스의 연산 성능과 자원 특성을 반영하여 인공지능 모델의 추론 성능을 예측하고 비교할 수 있는 논리적인 자원 단위이며, 인공지능 서비스를 실행할 때의 실질적인 성능을 반영하기 위해 인공지능 워크로드 기반의 벤치마크를 수행하여 산출된다. 실험을 통해 엣지 디바이스에서 리소스 셀 산출을 위한 벤치마크의 정확도를 분석한다. 또한 전력 효율과 분산 환경에서의 성능 특성을 평가하며, 실제 엣지 컴퓨팅 환경에서의 실험을 통해 제안한 리소스 셀 구조가 효과적인 자원 관리 방안임을 검증한다.

본 논문의 구성은 다음과 같다. 2장에서는 엣지 컴퓨팅 및 벤치마크 관련 연구를 소개하고, 3장에서는 리소스 셀 구조의 정의와 리소스 셀 산출을 위한 벤치마크 방법을 설명한다. 4장에서는 리소스 셀 구조의 구현 및 실험 결과를 제시하며, 5장에서는 성능 평가를 수행한다. 마지막으로 6장에서는 본 논문의 결론과 향후 연구 방향을 제시한다.

2. 관련 연구

2.1 엣지 컴퓨팅(Edge Computing)

엣지 컴퓨팅은 데이터 처리를 중앙의 클라우드 서버가 아닌, 데이터가 생성되는 지점 근처의 디바이스에서 수행하는 컴퓨팅 패러다임이다. 이는 클라우드 컴퓨팅의 높은 지연시간과 개인정보 유출 등의 문제를 해결하고자 제안된 방식이다[5]. 기존의 클라우드 컴퓨팅은 강력한 중앙 연산 자원을 바탕으로 거대한 인공지능 모델을 학습하고 추론하는 데 적합하지만, 사용자와의 물리적 거리 및 네트워크 대역폭 부족으로 인해 실시간성이 중요한 서비스에는 한계를 보인다. 반면, 엣지 컴퓨팅은 사용자 디바이스 또는 디바이스 주변의 엣지 노드에서

데이터를 실시간으로 처리함으로써, 낮은 지연 시간과 데이터 보안과 같은 이점을 갖는다[6]. 이러한 구조적 차이로 인해, 엣지 컴퓨팅은 클라우드와는 다른 연구 방향과 기술적 요구사항을 가진다. 엣지 환경은 일반적으로 연산 성능, 메모리 용량, 전력 공급 등 자원이 제한적이며, 이기종 디바이스들이 혼재되어 있는 것이 특징이다. 이에 따라 엣지 컴퓨팅 연구는 자원 제약성을 극복하기 위해 다음과 같은 방향으로 발전하고 있다. 첫째, 경량화된 인공지능 모델을 설계하여 연산량을 줄이고, 소형 디바이스에서도 실행 가능한 형태로 모델을 최적화하는 연구가 진행되고 있다. 둘째, 복잡한 연산은 엣지 클러스터나 가까운 엣지 서버로 오프로딩(offloading)하여 분산 처리하는 기법이 개발되고 있다. 셋째, 데이터의 수집과 처리를 로컬에서 수행함으로써 데이터 보안성을 높이는 연구도 활발히 이루어지고 있다[7].

2.2 엣지 인공지능(Edge Artificial Intelligence)

엣지 인공지능은 클라우드가 아닌 엣지 디바이스에서 인공지능 서비스를 수행함으로써, 지연을 최소화하고 실시간 처리가 요구되는 응용 분야에서 핵심적인 역할을 한다. 그러나 엣지 컴퓨팅 환경은 연산 성능, 메모리 용량, 전력 공급 등 자원이 제한적이므로, 경량화된 모델과 분산 처리와 같은 기술이 요구된다[8]. 이러한 분산 처리의 핵심은 자원 효율성을 극대화하는 데 있으며, 다수의 이기종 디바이스로 구성된 환경에서는 통신 비용, 디바이스별 연산 성능, 메모리 상태 등을 종합적으로 고려한 태스크 분산 전략이 중요하다[9]. 예를 들어, [10]에서는 CNN(Convolution Neural Network) 계열 모델인 AlexNet을 계층별로 분할하고, 통신 비용을 최소화하도록 오프로딩을 최적화하였다. [11]에서는 각 레이어의 연산량, 에너지 소비, 출력 크기에 기반해 분할 지점을 동적으로 결정하는 방식이다. [12]에서는 CNN의 피처맵을 타일 단위로 분할하여 중복 연산을 줄이고, 모델을 파이프라인(pipeline) 구조로 분산 처리하는 방식을 제안하였다. 또한, [13]에서는 엣지 서버에서 콘텐츠 캐싱(content caching), 오프로딩, 자원 관리를 통합적으로 고려하고, 이를 심층 강화학습(Deep Reinforcement Learning, DRL)으로 최적화한 접근법들을 분석하고 있다.

2.3 인공지능 워크로드 기반 벤치마크

디바이스의 성능을 이론적으로 분석하는 루프라인(Roofline) 모델은 응용 프로그램의 연산 강도(Operational Intensity, OI)와 디바이스의 메모리 대역폭, 연산 성능 한계를 결합하여 해당 모델이 연산 병목(compute-bound)인지 메모리 병목(memory-bound)인지를 시각적으로 판단할 수 있게 한다. 하지만 캐시 구조, 전력 수준 등 다양한 요인이 성능에 영향을 주기 때문에 실제 성능과 차이가 발생한다[14,15].

인공지능 시스템은 제공하고자 하는 서비스와 환경에 맞게 모델, 데이터 세트, 프레임워크, 아키텍처 등의 다양한 요소를 조합하여 다양화되었다. 이러한 환경에서 특정한 모델이나 아키텍처에 의존적인 벤치마크 시나리오는 정확한 추론 성능을 측정하여 비교하기 어렵다. 그러므로 벤치마크는 대표적인 워크로드를 사용하며 아키텍처에 독립적이고 다양한 환경에서도 그 결과를 재현 가능할 수 있어야 한다[16,17].

표 1 인공지능 워크로드 기반 벤치마크 비교
Table 1 Comparison Between AI Workload-based Benchmarks

Benchmark	Representative Models		Frameworks	Target
	Vision	Language		
MLPerf Inference [17]	ResNet, MoblieNet	GNMT	TensorFlow, PyTorch ONNX	Server, Desktop Mobile, Edge Device
AIoT Bench [18]	ResNet, Inception DenseNet, MobileNet MnasNet	-	TensorFow Pytorch	Mobile Edge Device
AI Benchmark [19]	MobileNet, Inception ResNet, SRCNN VGG	LSTM	TensorFlow, PyTorch ONNX, NNAPI	Mobile Desktop
DeepEdgeBench [20]	MobileNet	-	TensorFlow, TensorRT RKNN-Toolkit	Edge Device
EdgeMark [21]	ResNet, MoblieNet AutoEncoder	LSTM GRU	TensorFlow Edge Impulse	Edge Device

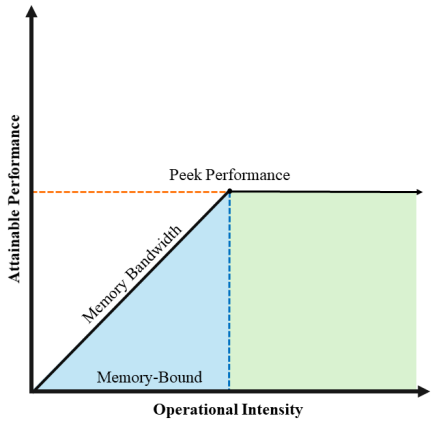

그림 2 루프라인 모델을 이용한 컴퓨팅 성능 표현 [14]
Fig. 2 Computational Performance Based on the Roofline Model

표 1은 엣지 컴퓨팅 환경에서 인공지능 워크로드의 성능을 평가하기 위해 널리 사용되는 대표적인 벤치마크들의 예시를 보여준다. 표에 제시된 각 벤치마크는 사용되는 모델, 인공지능 프레임워크, 그리고 하드웨어를 기준으로 구분된다. 인공지능 워크로드 기반의 벤치마크는 주로 이미지 인식과 객체 검출 같은 비전 모델(Vision)과 언어 모델(Language)을 대상으로 수행된다.

표 1에서 Frameworks는 해당 벤치마크가 의존하는 소프트웨어 라이브러리를 의미하며, 해당 프레임워크에 포함하는 하위 라이브러리들을 포괄한다. 예를 들어, TensorFlow는 기본 버전뿐만 아니라 TensorFlow Lite와 TensorFlow Lite Micro를 포함하는 상위 프레임워크로 정의된다. Target은 벤치마크의 실행 대상이 되는 하드웨어 환경을 의미한다. 이때 Mobile은 일반적인 스마트폰 환경을 지칭하며, Edge Device는 Moblie을 제외한 ARM 기반의 마이크로컨트롤러(Micro Controller Unit, MCU)와 Raspberry Pi와 같은 단일 보드 컴퓨터(Single Board Computer, SBC)를 포함하는 디바이스들을 의미한다.

엣지 디바이스 대상 벤치마크에서는 ResNet[22]과 MobileNet[23]이 대표적인 기준 모델로 사용된다. ResNet은 높은 연산량을 가지며 Convolution 연산이 주를 이루는 연산 집약적인 모델인 반면, MobileNet은 Depthwise Convolution과 Pointwise 연산으로 구성되어 연산량은 적지만 상대적으로 높은 메모리 접근량이 많은 메모리 집약적인 모델이다. 이러한 구조적 대비는 엣지 디바이스에서 연산 성능과 메모리 시스템 효율을 구분하여 평가할 수 있게 한다. 본 논문에서는 벤치마크의 대표성과 재현성을 고려하여 인공지능 모델과 프레임워크를 선정하였으며, 이에 대한 내용은 3.4에서 자세히 설명한다.

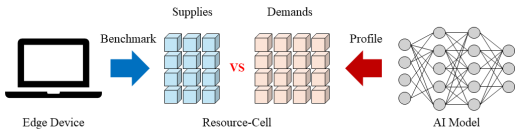

그림 3 리소스 셀 구조의 개념
Fig. 3 Concept of Resource-Cell Structure

3. 엣지 인공지능을 위한 리소스 셀 구조

3.1 리소스 셀의 개념

그림 3은 리소스 셀을 활용한 인공지능 서비스 모델의 핵심 개념을 나타낸다. 디바이스 연산 성능은 CPU 코어의 수와 클럭 속도를 기반으로 이론적인 수치를 계산할 수 있다. 하지만 실제 환경에서는 캐시 구조, 메모리 대역폭, 내부 버스 지연 등 다양한 요인으로 인해 성능 저하가 발생한다. 심지어 동일한 디바이스 내에서도 실행 시점이나 시스템 부하의 수준에 따라 동일한 태스크라도 성능 차이가 나타날 수 있다. 결국 성능은 디바이스의 하드웨어 구성에 따라 달라지며, 이론적 성능과 실제 측정 성능 간에는 차이가 존재한다.

한편, 인공지능 모델은 다수의 파라미터와 계층으로 구성된 연산 구조를 통해 입력 데이터를 처리하여 출력값을 생성하며, 실시간 추론을 보장하기 위해 일정 수준 이상의 연산 자원을 요구한다. 이때 모델의 연산 복잡도와 크기는 FLOPs 또는 파라미터의 수와 같은 수치로 표현될 수 있다. 하지만 동일한 계산량을 가지더라도 모델의 구조에 따라서 추론 시간의 차이가 발생한다.

본 논문에서는 벤치마크를 통해 엣지 디바이스가 보유한 연산 자원을 단위로 정량화하고, 인공지능 모델의 구조를 분석하여 해당 모델이 요구하는 리소스 셀의 수를 산출한다. 이후, 디바이스가 제공할 수 있는 양과 모델이 요구하는 양을 비교함으로써, 엣지 환경에서 보다 효율적인 자원 분배 및 인공지능 서비스 배치를 제안한다.

3.2 리소스 셀의 구성요소

그림 4는 엣지 디바이스 내에서 리소스 셀이 구성되고 분포되는 방식을 보여준다. 리소스 셀은 엣지 네트워크에 포함된 각 디바이스의 연산 성능을 정량적으로 표현하기 위한 기본 단위로 정의되며, 디바이스마다 서로 다른 개수의 리소스 셀을 가진다. 일반적으로 리소스 셀의 개수가 많을수록 해당 디바이스는 더 높은 처리 성능을 가지며, 반대로 적을수록 더 낮은 성능을 나타낸다.

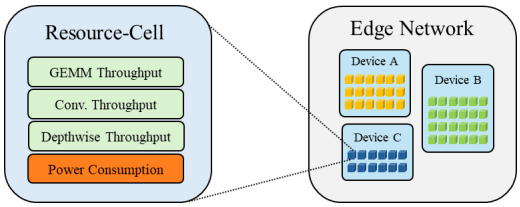

그림 4 리소스 셀의 구성 요소
Fig. 4 Consist of Resource-Cell

각 리소스 셀은 GEMM Throughput, Convolution Throughput, Depthwise Throughput, 그리고 Power Consumption의 네 가지 구성 요소로 정의된다. GEMM Throughput은 하나의 리소스 셀이 수행할 수 있는 GEMM(General Matrix Multiplication) 연산의 처리 성능을 의미하며, 연산 강도에 따라 값이 달라질 수 있다. Convolution Throughput은 일반적인 Convolution 연산에 대한 처리 성능을 의미하며, Depthwise Throughput은 Depthwise Convolution 연산에 대한 처리 성능을 의미한다. Power Consumption은 하나의 리소스 셀이 소비하는 전력량을 나타내며, 디바이스의 전체 전력 소비량을 해당 디바이스가 보유한 리소스 셀의 수로 나눈 값으로 정의된다. 이때 모든 리소스 셀이 소비하는 전력의 총합은 디바이스의 전체 전력 소비량과 일치하도록 구성된다.

GEMM, Convolution, Depthwise Convolution의 세 가지 연산자 중 GEMM은 행렬 곱셈을 수행하는 연산자로, 높은 연산량 대비 상대적으로 낮은 메모리 접근 특성을 가지므로 가장 높은 연산 강도(Operational Intensity, OI)를 나타낸다. 반면 Depthwise Convolution은 연산량에 비해 메모리 접근 비중이 매우 커 낮은 OI를 보인다. 일반적인 Convolution 연산은 이 두 연산자의 중간적인 특성을 가지며, 중간 수준의 OI를 갖는다.

본 논문에서는 인공지능 모델의 추론 성능을 효율적으로 예측하기 위해, GEMM, Convolution, Depthwise Convolution의 세 가지 연산자를 최소 분류 단위로 정의한다. 또한, 모든 디바이스의 GEMM Throughput 값이 동일한 OI 조건에서 일치하도록 정의한다. 세 가지 연산자에 대한 처리 성능은 GEMM 연산자를 기준으로 정규화되며, 본 논문에서 사용되는 리소스 셀의 개수는 GEMM 벤치마크를 통해 계산된 리소스 셀의 개수로 정의한다.

3.3 엣지 디바이스의 리소스 구조

그림 5는 엣지 디바이스가 보유한 자원을 표현한 리소스 테이블(Resource Table)을 보여준다. 리소스 테이블은 각 디바이스의 현재 상태를 체계적으로 파악하고,

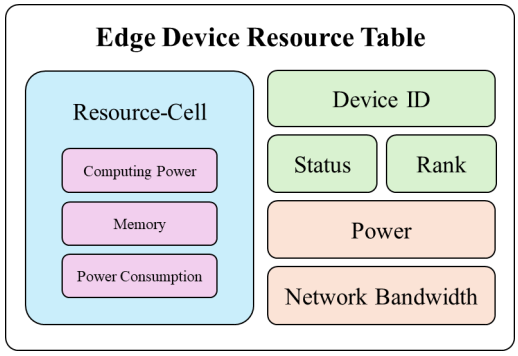

그림 5 리소스 셀이 적용된 디바이스 리소스 테이블
Fig. 5 Device Resource Table Based on the Resource-Cell

분산 처리 환경에서 작업 스케줄링과 자원 할당을 수행하기 위한 기초 데이터로 활용된다. 테이블은 Device, Resource-Cell, Environment의 세 가지 카테고리로 구성된다.

Device는 분산 환경에서 각 디바이스의 기본 정보를 의미한다. Device ID는 네트워크 내 디바이스를 구분하기 위한 고유 식별자이다. Status는 디바이스의 현재 동작 상태를 의미하며, Ready, Waiting, Running의 세 가지 상태로 구분한다. Ready는 태스크를 입력받을 수 있는 유휴(IDLE) 상태를 의미하며, Waiting은 작업이 할당된 이후 다음 입력을 대기하는 상태를 나타낸다. Running은 디바이스가 현재 입력된 데이터를 처리 중인 상태를 의미한다. Rank는 네트워크 내에서 디바이스의 역할과 우선순위를 나타내며, Rank 0은 서버, 그 외의 디바이스는 클라이언트로 구분된다.

Resource-Cell은 현재 디바이스가 가용한 자원을 리소스 셀 단위로 표현한다. 리소스 셀은 벤치마크를 통해 계산된 셀의 개수와 각 구성 요소들을 포함한다. Computing Power는 디바이스가 보유한 리소스 셀의 개수를 기반으로 계산된 처리 성능을 나타내며, GEMM Throughput, Convolution Throughput, Depthwise Throughput을 포함한다. Memory는 디바이스의 메모리 크기를 의미하며, 해당 디바이스가 수행 가능한 인공지능 모델의 크기를 비교하는 데 사용된다. Power Consumption은 디바이스의 전력 소비량을 나타내며, 디바이스의 지속 시간 분석을 위한 주요 지표로 활용된다.

Environment는 디바이스의 추론 작업 환경에 영향을 미치는 외부 요인들을 나타낸다. Network Bandwidth는 현재 연결된 엣지 서버 또는 다른 엣지 디바이스와의 통신 대역폭을 의미하며, 본 실험에서는 Wi-Fi

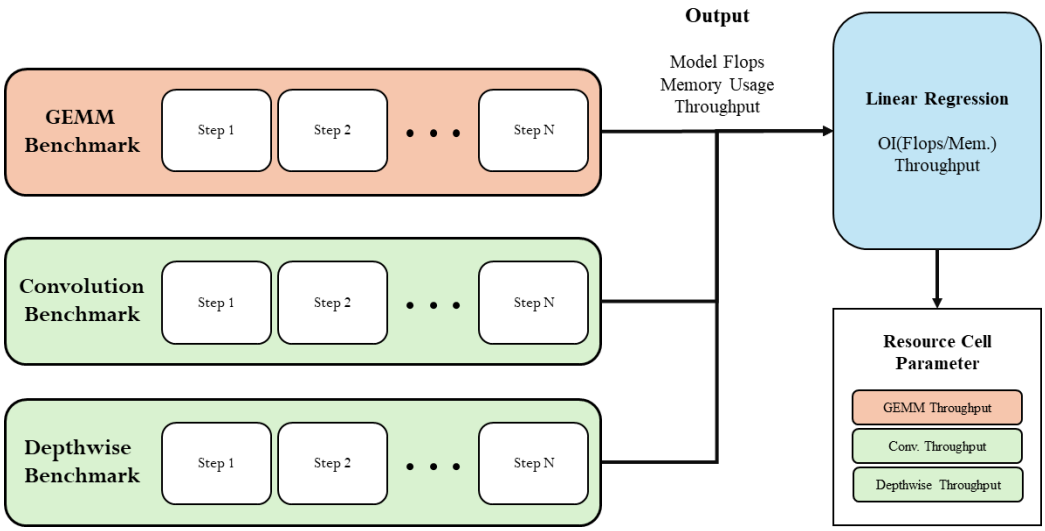

그림 6 리소스 셀 산출을 위한 벤치마크 구성
Fig. 6 Benchmark Suite for Resource-Cell Calculation

(IEEE 802.11) 환경에서 측정된 전송 속도를 사용한다. 해당 대역폭 정보는 분산 스케줄링 전반에서 중요한 파라미터로 활용된다. Power는 디바이스의 전압과 전류 정보를 포함하며, 무선 환경에서 동작하는 디바이스의 경우 이를 기반으로 잔여 전력량을 추정할 수 있다.

3.4 리소스 셀 산출을 위한 벤치마크

인공지능 워크로드를 수행함에 있어 디바이스의 성능은 연산 병목과 메모리 병목에 의해 제약을 받는다. 이러한 병목의 영향 정도는 워크로드의 특성에 따라 상이하게 나타나며, 단일한 벤치마크만으로는 다양한 실행 상황을 모두 고려하기 어렵다. 이에 본 논문에서는 리소스 셀 계산을 위해 서로 다른 연산자로 구성된 모델을 사용하여 벤치마크를 수행한다. 또한 각 벤치마크는 연산량과 메모리 사용 패턴이 단계적으로 변화하도록 설계된다.

그림 6은 이러한 벤치마크의 전체 구조를 보여준다. 각 모델은 단계별로 연산량과 메모리 접근량을 증가시키며, 해당 실험 결과를 바탕으로 처리량과 OI의 관계를 선형 회귀 모델로 학습한다. OI는 총 연산량을 메모리 사용량으로 나눈 값으로, 모델이 얼마나 연산 집약적인 특성을 가지는지를 나타내는 지표이다. 학습된 선형 모델은 리소스 셀에서 각 연산자에 대응하는 처리 성능을 계산하는 데 활용된다. 이를 통해 리소스 셀은 입력된 연산자의 OI에 따라 처리 성능이 변화하는 적응형 구조를 가진다.

본 논문에서 메모리 사용량의 절대값이 아닌 OI를 기반으로 모델링을 수행한 이유는, 실제 배포 환경에서 서비스 모델의 규모가 벤치마크 모델보다 훨씬 크고, 특정 아키텍처에 종속되지 않아야 하므로, 메모리의 절대적 사용량 대신 OI를 적용하여 선형 모델을 학습함으로써 아키텍처 독립성을 확보하였다.

3.5 인공지능 모델 프로파일러

리소스 셀을 이용하여 디바이스의 성능을 예측하기 위해서는, 디바이스가 보유한 리소스 셀의 개수뿐만 아니라 인공지능 모델이 요구하는 리소스 셀의 개수를 정량적으로 산출해야 한다. 이를 위해 대상 인공지능 모델을 계층별로 분해하고, 각 계층의 구조적 파라미터를 기반으로 연산자의 종류와 연산량을 계산한다.

연산자 분류는 계층의 연산 구조를 기준으로 수행된다. 예를 들어, 선형 변환을 수행하는 계층이라 하더라도 커널(kernel)을 사용하는 경우에는 Convolution 연산으로 분류하고, 커널을 사용하지 않는 경우에는 GEMM 연산으로 분류한다. 또한 Convolution 연산의 경우, 입력 채널 수와 그룹 수가 같은 경우에는 Depthwise Convolution으로 구분한다. 그 외에는 일반적인 Convolution으로 분류하여 연산량을 계산한다.

각 계층에서 요구되는 메모리 사용량은 해당 계층의 입력 데이터에 대한 읽기(read)와 출력 데이터에 대한 쓰기(write)의 합으로 산정한다. 이때 캐시 재사용이나 하드웨어 종속적인 메모리 최적화는 고려하지 않으며, 이를 통해 아키텍처 독립성을 유지하고 보수적인 성능 예측이 가능하게 한다.

4. 구현 및 실험

4.1 개발 환경

리소스 셀의 구조를 적용하기 위하여 Raspberry Pi를 실험 대상으로 사용하였다. Raspberry Pi는 리눅스(Linux) 기반의 싱글보드 컴퓨터로 ARM 기반의 프로세서를 탑재하고 있으며, 범용성을 바탕으로 다양한 분야에서 활용되고 있는 엣지 디바이스다. 본 연구에서는 이기종 디바이스 환경을 고려하여 Raspberry Pi 3B+, 4B, 5B 모델을 사용하였다. 각 디바이스는 동일한 운영체제를 사용하나 CPU 아키텍처와 성능, 메모리 대역폭 및 크기가 상이하기 때문에 추론 성능에서 차이를 보인다. 또한 실험 중 각 디바이스의 전력 소모량을 측정하기 위해 INA219 전류 센서를 사용하였다. 표 2는 Raspberry Pi의 버전별 성능을 보여준다.

표 2 Raspberry Pi 버전별 성능[24-26]
Table 2 Performance Across Raspberry Pi Version

Model	Processor	Memory
Raspberry Pi 3 B+	1.4GHz ARM Cortex-A53 MP4	1GB LPDDR2-500 SDRAM
Raspberry Pi 4 B	1.5GHz ARM Cortex-A72 MP4	2GB LPDDR4-3200 SDRAM
Raspberry Pi 5 B	2.4GHz ARM Cortex-A76 MP4	4GB LPDDR4X-4266 SDRAM

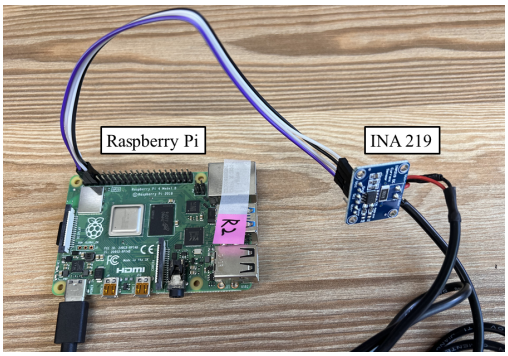

그림 7 Raspberry Pi와 전류 센서 구성
Fig. 7 Raspberry Pi and Current Sensor Configuration

그림 7은 실험에 사용된 Raspberry Pi와 연결된 전류 센서 INA219의 연결 구성을 보여준다. 전류 센서는 Raspberry Pi에 공급되는 전원에 연결되어 실시간으로 전류 소모량을 측정할 수 있다. INA219 센서는 션트 저항(shunt resistor)을 사용하여 저항 양단의 전압과 저항 값을 이용해 전류량을 계산한다[27].

4.2 벤치마크 워크로드 파라미터

각 벤치마크는 Step 1부터 Step 10까지 총 10단계로 구성되며, 각 단계에서 동일한 구조의 모델에 대해 벤치마크를 수행한다. 이 과정에서 모델의 크기를 단계적으로 증가시켜, 연산량과 메모리 접근량이 점진적으로 증가하도록 설계하였다. 이를 통해 연산 집약도와 메모리 집약도가 서로 다른 다양한 실행 조건을 체계적으로 반영할 수 있다.

표 3은 벤치마크를 구성하는 세 가지 연산자 기반 모델에 대해 파라미터 수와 연산량을 요약하여 보여준다. 표 3에서는 각 모델의 시작 단계인 Step 1과 종료 단계인 Step 10에 해당하는 파라미터 수를 제시한다. 모든 모델은 연산 특성을 명확히 분석하기 위해 단일 계층(single-layer) 구조로 구현되었다.

Parameter는 각 벤치마크 모델의 입출력 크기, 채널 수, 커널 크기 등 구조적 설정값을 나타낸다. FLOPs는 각 단계에서 수행되는 총 연산량을 나타내며, 모든 연산

표 3 벤치마크 모델 파라미터와 자원 요구량
Table 3 Benchmark Model Parameters and Resource Requirements

Model	Step 1			Step 10			Nonlinear
	Parameter	FLOPs	Memory	Parameter	FLOPs	Memory	
GEMM	M=256, K=256, N=256	33.5M	1MB	M=1024, K=1024, N=1024	2,147.5M	16.8MB	ReLU Softmax
Convolution	Cin=6, Cout=12, H=224, W=224, K=3	65M	3.6MB	Cin=24, Cout=48, H=224, W=224, K=3	1,040.5M	14.5MB	ReLU, BatchNorm, Pooling
Depthwise	C=8, H=64, W=64, K=3	1.3M	0.4MB	C=96, H=256, W=256, K=3	113.3M	50.3MB	ReLU, BatchNorm, Pooling

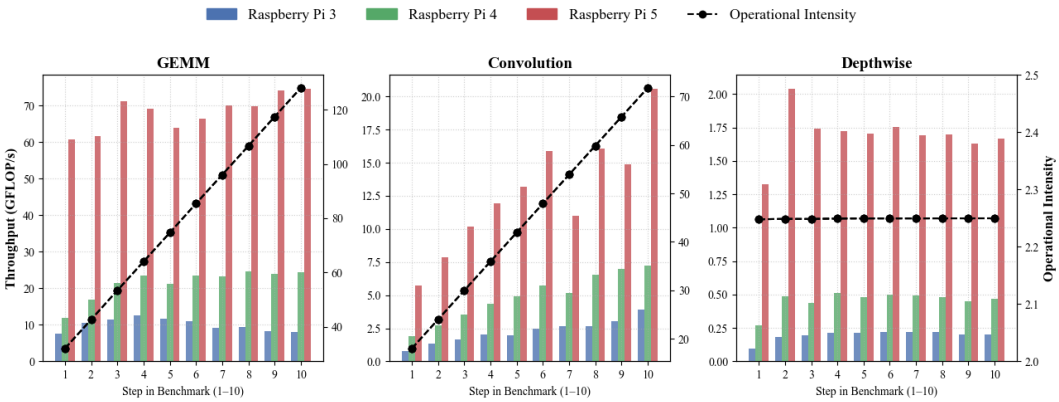

그림 8 Raspberry Pi에서 수행한 벤치마크 결과
Fig. 8 Benchmark Results Across Three Raspberry Pi Versions

은 float32 정밀도를 기준으로 계산된다. Memory는 해당 모델의 메모리 접근량을 의미하며, 실제 실행 과정에서 발생할 수 있는 메모리 접근 비용을 고려하여 보수적으로 산정하였다.

또한 Nonlinear 연산자는 모델의 출력층에 적용되는 비선형 함수로, 연산량 자체는 상대적으로 작지만, 계층 간 데이터 이동과 메모리 접근으로 인해 발생하는 지연 효과를 반영하기 위해 포함하였다.

4.3 리소스 셀 계산

그림 8은 리소스 셀 산출을 위해 사용된 3종류의 Raspberry Pi에서 수행된 결과를 보여준다. 각 디바이스에서 3가지의 벤치마크를 수행하였고, 각 단계별 평균 처리량과 모델의 연산 강도(OI)를 보여준다. 그래프는 벤치마크 하나에서 각 Raspberry Pi 버전 별 처리량과 모델의 OI를 같이 나타낸다.

GEMM은 세 가지 모델 중 가장 연산 집약적인 연산자로 가장 처리량이 높게 나타난다. OI는 단계에 따라 선형적으로 증가한다. 처리량은 단계에 따라 소폭 증가하거나 유지되는 형태로 나타난다. 이는 연산성능이 최대치(peak)를 이미 기록하고, 메모리 사용량이 증가하여 메모리 병목으로 인한 성능 저하로 분석된다.

Convolution 연산은 중간 수준의 OI를 가지며, 평균적으로 GEMM 대비 약 55% 수준의 OI를 나타낸다. 이에 따라 처리량 또한 GEMM 대비 평균 약 18.2% 수준으로 측정된다. GEMM에 비해 절대적인 OI 값이 낮기 때문에, 메모리 병목이 상대적으로 이른 단계에서 발생하며, 이로 인해 OI 증가에 비례하여 처리 성능이 선형적으로 증가하는 양상을 보인다.

Depthwise Convolution 연산은 세 가지 연산자 중 가장 낮은 OI 특성을 가진다. Depthwise Convolution은 엣지 환경에서의 효율적인 추론을 위해 채널별로 독립적인 필터를 적용하여 입력 채널과 출력 채널의 수가 동일한 구조이다. 이러한 구조적 특성으로 인해 모델의 크기가 증가할수록 연산량보다 메모리 사용량이 비례적으로 증가하게 되며, 그 결과 OI는 전 구간에서 거의 일정한 값을 유지한다. Raspberry Pi 환경에서는 Step 2에서 처리 성능의 최대치를 달성한 이후, 성능이 소폭 감소하거나 유지되는 경향을 보인다. 이는 모델 규모가 증가함에 따라 메모리 접근 비용이 급격히 증가하여, 비교적 낮은 단계에서 메모리 병목이 발생하기 때문으로 분석된다.

표 4 각 연산자별로 계산된 리소스 셀의 수
Table 4 Resource-Cell Values Computed for Each Operator

Operator	Avg. OI	RPi 3B+	RPi 4B	RPi 5B
GEMM	79.99	998.21	2146.84	6814.79
Convolution	44.91	224.22	452.05	1187.91
Depthwise	2.25	19.57	45.91	169.76

표 4는 이러한 벤치마크 결과를 바탕으로 계산된 Raspberry Pi(RPi)의 연산자별 리소스 셀 개수를 보여준다. 리소스 셀 산출 모델에는 각 벤치마크에서 측정된 평균 OI 값을 적용하였으며, 이는 특정 단계가 아닌 디바이스의 평균적인 연산 성능 특성을 반영하기 위한 것이다. 본 논문에서 하나의 리소스 셀은 10MFLOPs/s의 처리량을 갖는 단위로 정의된다.

4.4 리소스 셀을 적용한 분산 처리 구조

본 논문에서는 엣지 환경에서 리소스 셀 기반의 효율

적인 분산 추론을 위해 MQTT(Message Queuing Telemetry Transport) 기반의 통신 구조를 구현하였다. 그림 9는 본 시스템의 전체 구조와 각 구성 요소 간의 역할 및 통신 흐름을 보여준다. 서버는 중앙에서 클라이언트에게 태스크 수행 명령과 관련 메타데이터를 MQTT 메시지 형태로 발행(publish)하며, 클라이언트가 주기적으로 발행하는 자원 상태 정보와 태스크 처리 결과를 구독(subscribe)하여 수신한다. 반대로 클라이언트는 서버의 태스크 명령과 메타데이터를 구독하고, 자신의 자원 상태 및 태스크 수행 결과를 주기적으로 발행한다. 실제 인공지능 모델의 입력 데이터나 중간 추론 결과와 같은 대용량 정보는 MQTT가 아닌 소켓 통신을 통해 송수신되며, MQTT 메시지에는 해당 데이터의 메타데이터만 포함된다. 서버는 연결된 클라이언트 디바이스의 자원 상태를 실시간으로 모니터링하며, 이를 기반으로 적절한 디바이스에 태스크를 분배한다. 또한 각 클라이언트로부터 수신된 태스크 처리 결과를 수집하고 후속 처리에 활용한다. 클라이언트는 두 가지 형태의 인공지능 워크로드를 수행한다. 첫째, 벤치마크를 위한 인공지능 워크로드를 통해 리소스 셀을 계산하고, 둘째, 서버로부터 분배받은 태스크를 기반으로 실제 인공지능 추론을 수행한다.

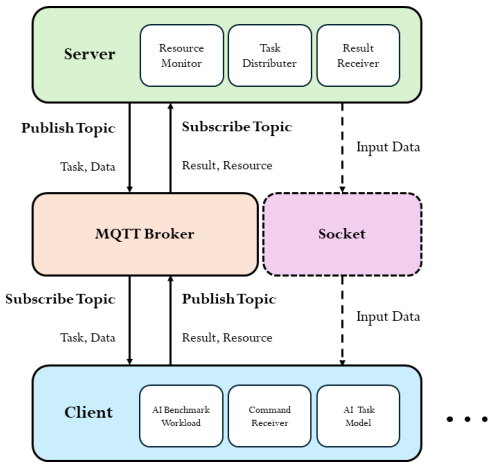

그림 9 리소스 셀을 적용한 MQTT 기반 분산 처리 구조

Fig. 9 MQTT-Based Distributed Processing Architecture with Resource-Cell

Client ID	IP	Task	GEMM	CONV	DEPTH	Bandwidth	Memory	Battery	Timestamp
rpi5	192.168.101.9	Ready	7382	1713	173	0.0	3449.4	100.0	2026-01-08 22:14:38
rpi4	192.168.101.2	Ready	2794	645	48	0.0	1454.4	100.0	2026-01-08 22:14:37
rpi3	192.168.101.12	Ready	875	312	20	0.0	455.7	100.0	2026-01-08 22:14:37

그림 10 MQTT 서버의 리소스 모니터링 실행화면

Fig. 10 Excution Screen of Resource Monitoring on the Server

그림 10은 서버에서 각 클라이언트의 리소스 상태를 모니터링하여 리소스 테이블로 출력하는 예시를 보여준다. 각 항목은 클라이언트 식별을 위한 Client ID, 통신 주소인 IP, 현재 태스크 상태를 나타내는 Task, 보유 리소스 셀 수를 나타내는 Resource-Cell, 데이터 통신 속도인 Bandwidth, 사용 가능한 메모리, 잔여 배터리로 구성된다.

5. 성능 평가

본 장에서는 리소스 셀 구조의 효용성을 검증하기 위해 다양한 시나리오를 적용하여 분석을 수행한다. 이를 위해 제안한 리소스 셀 표현 방식이 실제 성능을 얼마나 정확하게 반영하는지 확인하고, 이를 기반으로 분산 처리 환경에서의 활용 가능성과 효율성을 평가한다.

먼저, 벤치마크를 통해 산출된 리소스 셀이 실제 인공지능 모델의 추론 시간을 얼마나 정확하게 예측할 수 있는지를 분석한다. 이를 통해 리소스 셀 기반 성능 예측의 타당성을 검증한다.

다음으로, 리소스 셀을 기준으로 엣지 디바이스에서

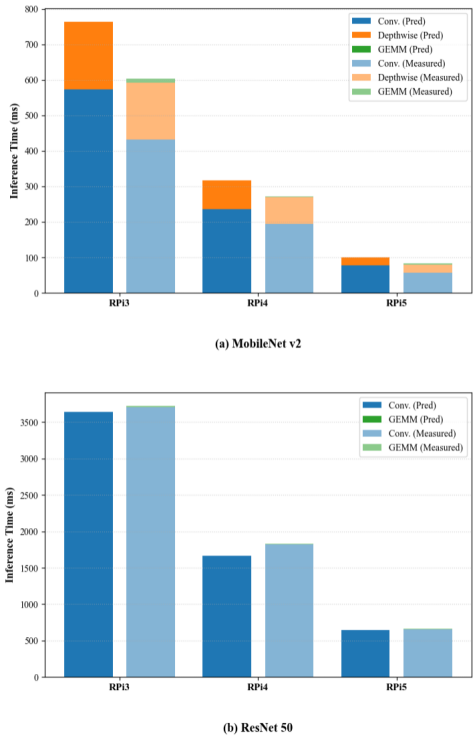

그림 11 리소스 셀 기반 추론 시간 예측의 신뢰도 평가

Fig. 11 Evaluation of Inference Time Prediction Reliability Based on Resource-Cell

소비된 전력이 어느 정도의 연산량을 처리할 수 있는지를 정량적으로 분석하여, 전력 효율 관점에서의 유용성을 평가한다.

마지막으로, 분산 처리 환경에서 리소스 셀 기반의 작업 분산 기법을 적용하여 로드 밸런싱 효율과 지연 시간에 미치는 영향을 분석함으로써, 제안한 구조의 실질적인 시스템적 효과를 확인한다.

5.1 리소스 셀 산출 신뢰도 평가

그림 11은 리소스 셀을 이용하여 인공지능 모델의 추론 시간을 예측한 결과를 보여준다. 추론 시간 예측은 대상 모델을 사전에 프로파일링한 후, 각 디바이스가 보유한 리소스 셀 정보를 기반으로 연산자별 추론 시간을 계산하고 이를 합산하는 방식으로 수행된다. 그림에서는 전체 추론 시간 중 각 연산자가 차지하는 비중을 나타내었다. 여기서 Pred는 리소스 셀 기반으로 계산된 예측 추론 시간을 의미하며, Measured는 해당 디바이스에서 실제로 측정한 실행 시간을 의미한다.

성능 평가를 위해 메모리 집약적인 모델인 MobileNet-V2와 연산 집약적인 모델인 ResNet-50을 사용하였다. 두 모델은 인공지능 워크로드 기반 벤치마크에서 대표적으로 활용되는 모델이다. 또한 상반된 연산 특성을 지니고 있어 다양한 성능 특성을 평가하기에 적합하다. 벤치마크 결과의 재현성을 확보하기 위해 PyTorch 라이브러리에서 제공하는 공식 구현 모델을 사용하였으며, 모든 실험에서 입력 데이터는 1×224×224 크기로 통일하였다.

(a) MobileNet-V2는 약 603 MFLOPs의 연산량을 요구하며, 연산량의 약 92.6%가 Convolution 연산, 약 6.8%가 Depthwise Convolution으로 구성되어 있다. 완전연결 계층을 구성하는 GEMM 연산은 전체 연산량의 약 0.5% 수준에 불과하다. 해당 모델의 추론 시간 예측 신뢰도는 약 77.7%로 나타났으며, 예측된 추론 시간이 실측값보다 다소 높게 나타났다. 이는 메모리 사용량을 보수적으로 계산함으로써, 메모리 병목으로 인한 처리 지연을 과대 추정하였기 때문이다. 연산자별로 살펴보면, Convolution 연산에서 약 30.5%, Depthwise Convolution 연산에서 약 9.35%의 오차가 발생하였다. GEMM 연산은 전체 비중이 작지만, 연산 과정에서의 메모리 병목 영향으로 인해 상대적으로 낮은 예측 신뢰도를 보였다.

(b) ResNet-50은 약 8,178 MFLOPs의 연산량을 요구하는 연산 집약적인 모델이다. 출력층을 제외한 대부분의 계층은 Convolution 연산과 이에 수반되는 비선형 함수로 구성되어 있으며, 실제 실행 시간의 약 99%가 Convolution 계층에서 소요된다. ResNet-50은 MobileNet-V2와 달리 연산 강도가 높은 특성을 가지므로 메모리 병목으로 인한 지연이 상대적으로 적게 나타난다. 이에 따라 전체 추론 시간 예측 신뢰도는 약 94.27%로 측정되었으며, Convolution 연산에 대한 예측 신뢰도 또한 이와 유사한 경향을 보인다.

5.2 리소스 셀 기반 전력 효율 분석

리소스 셀의 전력 소모량은 디바이스의 유휴 상태 기본 소비 전력을 기준으로, 리소스 셀의 활성화 정도와 사용 시간에 비례하여 증가하는 것으로 모델링하였다. 이때 리소스 셀의 사용량 증가는 CPU 사용률 증가와 선형적으로 비례한다고 가정하였다. 즉, CPU 사용률이 높아질수록 더 많은 리소스 셀이 동시에 활성화되어 에너지를 소비한다. 그러므로 리소스 셀의 전력 소비 모델은 다음과 같이 정의한다.

$$Power\ Consumption = (P_{Idle} + P_{rc} \times n) \times t \qquad (1)$$

식 (1)에서 P_{Idle}은 유휴 상태일 때의 전력 소비량을 의미한다. P_{rc}는 리소스 셀 하나가 활성화될 때 사용되는 전력 소비량을 의미하며, n은 활성화된 리소스 셀의 수, t는 활성화 시간을 의미한다.

본 실험에서는 디바이스별 리소스 셀 수와 측정된 CPU 사용률을 바탕으로 각 시점에서 활성화된 리소스 셀 수를 추정하였다. 이후 리소스 셀 활성도를 점진적으로 증가시키면서 소비되는 전력량을 측정하였다. 측정된 전력 소비량과 CPU 사용량 간의 상관관계는 선형회귀를 이용해 모델링하였다. 손실함수(Loss Function)는

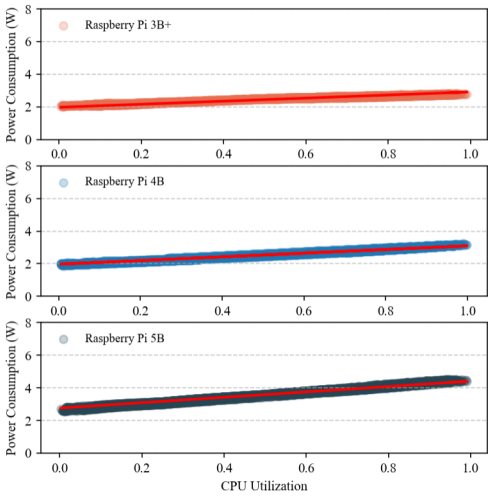

그림 12 Raspberry Pi 버전별 CPU 사용량 대비 전력 소모량

Fig. 12 CPU Utilization vs. Power Consumption Across Raspberry Pi Versions

평균제곱오차(Mean Square Error, MSE)를 기반으로 계산되었다.

그림 12는 세 종류의 Raspberry Pi를 대상으로 리소스 셀 사용량에 따른 전력 소비 특성을 측정하고, 그 결과에 대해 회귀 분석을 수행한 결과를 나타낸다. X축은 CPU 사용률(0-1)을 나타내며, Y축은 단위 시간당 소비된 전력(W)을 나타낸다. 실험 결과, CPU 사용률이 증가함에 따라 전력 소비가 전반적으로 선형적인 증가 경향을 보이는 것을 확인할 수 있다.

Raspberry Pi 5B는 CPU 사용률 증가에 따른 전력 소비 증가율이 가장 크게 나타나, 높은 연산 성능을 제공하는 대신 상대적으로 높은 전력 소모 특성을 가지는 것으로 나타났다. 반면 Raspberry Pi 3B+는 전력 증가율이 완만하게 나타나며, 낮은 성능 특성과 낮은 전력 소비 특성을 동시에 보인다. 이러한 결과는 리소스 셀 활성화 정도와 전력 소비 간의 선형 관계를 가정한 본 논문의 전력 모델이 실제 엣지 디바이스 환경을 합리적으로 반영하고 있음을 보여준다.

그림 13은 Raspberry Pi에서 제안한 리소스 셀 구조를 적용하여 전력 효율을 분석한 결과를 보여준다. 본 분석은 제한된 전력 조건에서 (a) 주어진 전력으로 얼마나 많은 연산 리소스를 활용할 수 있는지와, (b) 동일한 전력을 사용하여 시스템이 얼마나 오랜 시간 동안 동작할 수 있는지를 평가하는 두 가지 시나리오로 구성된다.

(a)는 전력 사용량에 따른 리소스 셀 활용 효율을 나타낸 그래프이다. 세 종류의 Raspberry Pi를 대상으로,

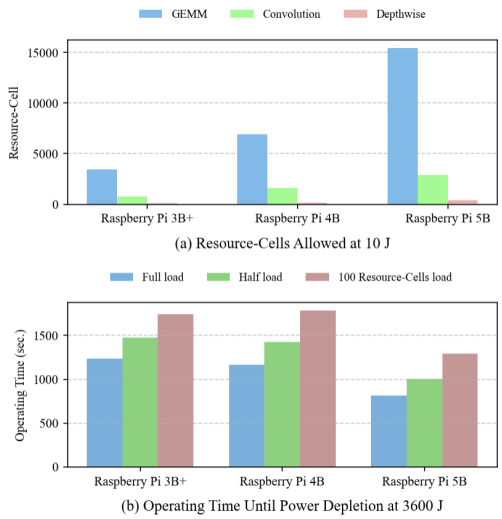

그림 13 Raspberry Pi 버전별 리소스 셀 전력 효율 분석
Fig. 13 Power Efficiency Analysis of Resource-Cells Across Raspberry Pi Versions

10J의 에너지가 주어졌을 때 활성화할 수 있는 리소스 셀의 개수를 비교하였다. x축은 디바이스와 리소스 셀을 구성하는 연산 유형(GEMM, Convolution, Depthwise Convolution)을 의미하며, y축은 사용 가능한 리소스 셀의 수를 의미한다. GEMM 연산을 기준으로 살펴보면, Raspberry Pi 3B+는 10J의 에너지로 약 3,424개의 리소스 셀에 해당하는 연산을 수행할 수 있다. Raspberry Pi 4B는 약 6,933개의 리소스 셀을 활성화할 수 있어 Raspberry Pi 3B+ 대비 약 202%의 효율을 보인다. Raspberry Pi 5B는 약 15,446개의 리소스 셀을 사용할 수 있으며, 이는 Raspberry Pi 3B+ 대비 약 451%에 해당하는 전력 효율이다. 이러한 결과는 전력이 제한된 환경에서 연산 성능이 우수한 디바이스일수록 더 많은 리소스를 효과적으로 활용할 수 있음을 확인할 수 있다.

(b)는 3,600J의 에너지가 주어졌을 때, 리소스 셀 활성화 수준에 따른 시스템의 지속 시간을 나타낸다. 각 디바이스에 대해 모든 리소스 셀이 활성화된 상태, 절반의 리소스 셀이 활성화된 상태, 그리고 평균적으로 100개의 리소스 셀만 사용하는 상태를 비교하였다. 모든 리소스가 활성화된 경우는 대규모 연산 작업을 대기 없이 수행하는 상황을 의미하며, 이 조건에서는 Raspberry Pi 3B+가 가장 긴 지속 시간을 보인다. 그러나 동일한 시간 동안 Raspberry Pi 5B는 Raspberry Pi 3B+ 대비 약 451%에 해당하는 연산량을 처리할 수 있다. 반면, 평균적으로 100개의 리소스 셀만 사용하는 경우는 시스템 지속 시간 중에서 처리 시간보다 대기 시간이 상대적으로 많은 시나리오를 의미한다. 이 경우 Raspberry Pi 3B+가 더 긴 시간 동안 동작하여 지속 시간 관점에서 효율적인 구조를 갖는 것으로 분석된다.

5.3 리소스 셀 기반의 작업 분산 성능 평가

본 실험에서는 MQTT 기반 네트워크에서 리소스 셀 기반의 작업 분배가 로드 밸런싱과 지연 시간에 미치는 영향을 분석한다. 서버가 각 디바이스에 입력 데이터를 송신하면 디바이스에 배포된 MobileNet-V2, ResNet-50 모델에서 추론을 수행하고 결과를 다시 서버로 전송하는 방식으로 동작한다. 실험에서는 FCFS(First Come First Served) 방식으로 작업을 분산하였고, 작업은 100개의 입력 데이터를 모두 추론할 때까지 수행되었다.

그림 14는 분산 처리 환경에서 작업을 분산하였을 때, 세 종류의 Raspberry Pi에서 처리한 총 추론 횟수를 나타낸다. Predict는 디바이스가 보유한 리소스 셀을 바탕으로 계산된 예상 작업 처리량을 의미한다. FCFS(2)는 한 번에 2개의 입력 데이터를 전송하는 방식, FCFS(Resource)는 가장 작은 리소스 셀을 보유한 디바이스가 1개의 입력 데이터를 전송받고, 리소스 셀을 더 많이 보유한 디바이스는 상대적인 성능에 비례하여

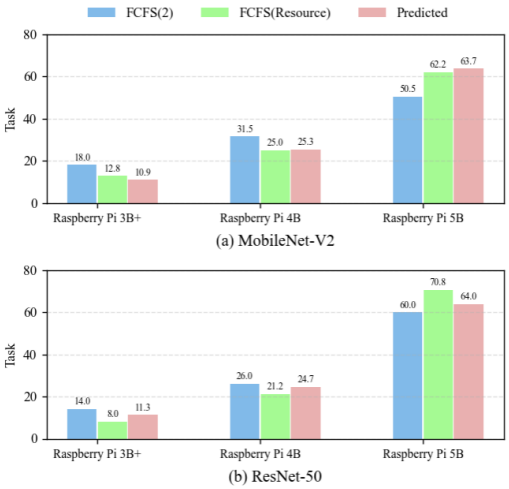

그림 14 분산 추론 전략에 따른 Raspberry Pi별 추론 횟수 비교

Fig. 14 Number of Inference Comparison Across Raspberry Pi Versions for Distributed Inference Strategies

더 많은 데이터를 입력받는 방식이다. 본 실험에서는 1:3:6 비율로 전송되었다.

MobileNet-V2를 분산 추론하는 과정에서 리소스 셀 기반의 방식은 리소스 셀 보유량으로 예측한 결과와 약 93%의 유사도를 갖는다. 동일한 크기로 데이터를 나눈 경우 성능이 비교적 낮은 Raspberry Pi에 작업이 과도하게 집중되는 현상이 나타났다. 반면에 ResNet-50을 대상으로 한 분산 추론에서는 약 78%의 유사도를 보인다. MobileNet-V2와 상반되게, 성능이 높은 Raspberry Pi에서 예상치보다 더 많은 작업을 수행하였다.

그림 15는 작업 분산 방식에 따른 MobileNet-V2, ResNet-50의 전체 처리 시간(Turnaround Time)을 비교한 결과를 나타낸다. FCFS(Resource) 방식이 전반적으로 낮은 지연 시간을 기록하며, 기존 FCFS 기반 스케줄링보다 우수한 성능을 보였다. MobileNet-V2에서는 4.7% 더 빠르게 작업을 수행하고, ResNet-50에서는 FCFS(2)와 리소스 셀 기반의 전략이 16.4% 더 빠르게 응답하는 것으로 측정되었다.

작업 분산 결과를 종합적으로 분석했을 때, MobileNet-V2에서는 리소스 셀 기반 방식은 수행 작업량을 비교적 정확하게 예측하였으나, 실제 처리 시간 단축 효과는 크지 않았다. 연산 집약적인 ResNet-50에서는 작업량 예측에서는 비교적 낮은 정확도를 보였으나, 실제 처리 시간 단축에서 유효함을 보인다. 이는 분산 추론 환경에서 통신 오버헤드를 제외하면, 실제 추론 성능이 연산 집약도에 따라 달라지기 때문으로 분석된다.

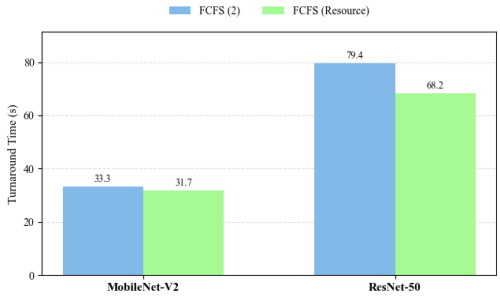

그림 15 분산 추론 전략에 따른 워크로드별 응답 시간 비교

Fig. 15 Turnaround Time Comparison Across Workloads for Distributed Inference Strategies

6. 결론 및 향후계획

본 논문에서는 이기종 엣지 컴퓨팅 환경에서 분산 인공지능 워크로드를 효율적으로 수행하기 위한 리소스 셀(Resource-Cell) 구조를 제안하였다. 기존의 단순한 연산 성능 기반 자원 비교 방식은 디바이스 간의 메모리 병목, 전력 제약, 그리고 워크로드 특성을 충분히 반영하지 못하는 한계가 있다. 이를 해결하기 위해 본 연구에서는 인공지능 워크로드 기반 벤치마크를 활용한 정량적 자원 표현 방식을 도입하였다.

제안한 리소스 셀은 GEMM, Convolution 그리고 Depthwise Convolution 연산에 대한 디바이스의 처리 성능과 전력 소모량을 추상화한 논리적 자원 단위이다. 각 연산자에 대한 처리 성능을 정량화하기 위해, 본 논문에서는 연산자별로 10단계로 구성된 벤치마크를 수행하고 이를 바탕으로 리소스 셀의 개수를 산출하는 방법을 제안하였다.

리소스 셀 구조의 구현 및 검증을 위해 Raspberry Pi 시리즈를 실험 대상으로 사용하여 디바이스별 리소스 셀의 개수를 정량화하고, 이에 따른 전력 소비 특성을 분석하였다. 또한 MQTT 기반의 분산 처리 시스템을 구성하여, 리소스 셀 정보를 기반으로 한 동적 작업 분배 기법을 적용하였다.

리소스 셀 기반 성능 예측의 신뢰도를 평가하기 위해 MobileNet-V2와 ResNet-50 모델을 사용하였다. 실험 결과, 연산 집약적인 모델인 ResNet-50에 대해서는 최대 94% 이상의 높은 추론 시간 예측 신뢰도를 확인하였다. 더불어 리소스 셀의 활성화 수준과 CPU 사용률 간의 선형 관계를 가정한 전력 소비 모델을 제안하였다. CPU 사용률이 증가함에 따라 전력 소비는 전반적으로

선형적인 증가 경향을 보였으며, 이를 통해 제한된 전력 조건에서 디바이스별 연산 효율과 동작 지속 시간을 분석할 수 있음을 확인하였다. 특히 고성능 디바이스인 Raspberry Pi 5B는 제한된 전력 환경에서 더 많은 연산 리소스를 활용할 수 있는 반면, Raspberry Pi 3B+는 상대적으로 낮은 전력 소모 특성으로 인해 장시간 동작에 유리한 특성을 보였다.

마지막으로 MQTT 기반의 분산 추론 환경에서 리소스 셀을 활용한 작업 분산 전략을 적용한 결과, 동일한 데이터 크기를 전송하는 FCFS 방식 대비 로드 밸런싱 효율이 향상되고 전체 처리 지연 시간이 감소하는 것을 확인하였다. 특히 연산 집약적인 모델인 ResNet-50에서는 16.4% 빠르게 작업을 완료하여 리소스 셀 기반 분산 전략이 실제 처리 시간 단축에 효과적임을 입증하였다.

종합적으로 본 논문은 리소스 셀 구조가 이기종 엣지 환경에서 인공지능 워크로드의 성능 예측, 전력 효율 분석, 그리고 분산 작업 스케줄링을 통합적으로 지원할 수 있는 유효한 자원 정량화 방식임을 입증하였다.

향후 연구에서는 리소스 셀 산출의 정밀도를 향상시키기 위해 더 세분화된 벤치마크 모델을 도입하고, 다양한 아키텍처 기반 디바이스를 추가하여 시스템의 확장성을 개선할 예정이다. 또한 네트워크 지연과 통신 오버헤드를 리소스 셀 모델에 포함함으로써, 더 정확한 성능 예측을 달성하고자 한다.

References

[1] J. Aljbour, T. Wilson, and P. Patel, “Powering Intelligence: Analyzing Artificial Intelligence and Data Center Energy Consumption,” *EPRI White Paper No. 3002028905*, May. 2024.

[2] W. Shi, J. Cao, Q. Zhang, Y. Li, and L. Xu, “Edge Computing: Vision and Challenges,” *IEEE Internet of Things Journal,* Vol. 3, No. 5, pp. 637-646, Oct. 2016.

[3] M. Satyanarayanan, “The Emergence of Edge Computing,” *Computer,* Vol. 50, No. 1, pp. 30-39, Jan. 2017.

[4] T. Taleb, K. Samdanis, B. Mada, H. Flinck, S. Dutta, and D. Sabella, “On Multi-Access Edge Computing: A Survey of the Emerging 5G Network Edge Cloud Architecture and Orchestration,” *IEEE Communications Surveys & Tutorials,* Vol. 19, No. 3, pp. 1657-1681, May. 2017.

[5] W. Yu, F. Liang, X. He, W. G. Hatcher, C. Lu, J. Lin, and X. Yang, “A Survey on the Edge Computing for the Internet of Things,” *IEEE Access,* Vol. 6, pp. 6900-6919, Nov. 2017.

[6] N. Abbas, Y. Zhang, A. Taherkordi, and T. Skeie, “Mobile Edge Computing: A Survey,” *IEEE Internet of Things Journal*, Vol. 5, No. 1, pp. 450-465, Feb. 2018.

[7] J. Chen and X. Ran, “Deep Learning with Edge Computing: A Review,” *Proc. of the IEEE,* Vol. 107, No. 8, pp. 1655-1674, 2019.

[8] S. S. Gill et al., “Edge AI: A Taxonomy, Systematic Review and Future Directions,” *Cluster Computing*, Vol. 28, No. 1, pp. 1-53, Oct. 2024.

[9] X. Wang and W. Jia, “Optimizing Edge AI: A Comprehensive Survey on Data, Model, and System Strategies,” *arXiv preprint,* arXiv:2501.03265, Jan. 2025.

[10] Y. Kang et al., “Neurosurgeon: Collaborative Intelligence Between the Cloud and Mobile Edge,” *ACM SIGARCH Computer Architecture News,* Vol. 45, No. 1, pp. 615-629, Apr. 2017.

[11] M. Zawish, L. Abraham, K. Dev, and S. Davy, “Towards Resource-aware DNN Partitioning for Edge Devices with Heterogeneous Resources,” *Proc. of the 2022 IEEE Global Communications Conference (GLOBECOM),* pp.5649-5655, 2022.

[12] Z. Zhao, K. M. Barijough, and A. Gerstlauer, “Deepthings: Distributed Adaptive Deep Learning Inference on Resource-Constrained IoT Edge Clusters,” *IEEE Transaction Computer-Aided Design of Integrated Circuits and Systems,* Vol. 37, No. 11, pp. 2348-2359, Nov. 2018.

[13] A. A. Ismail, N. E. Khalifa, and R. A. El-Khoribi, “A Survey on Resource Scheduling Approaches in Multi-Access Edge Computing Environment: A Deep Reinforcement Learning Study,” *Cluster Computing*, Vol. 28, No. 184, Jan. 2025.

[14] S. Williams, A. Waterman, and D. Patterson, “Roofline: An Insightful Visual Performance Model for Multicore Architectures,” *Communications of the ACM,* Vol. 52, No. 4, pp. 65-76, Apr. 2009.

[15] Victoria Caparrós Cabezas and Markus Püschel, “Extending the Roofline Model: Bottleneck Analysis with Microarchitectural Constraints,” *Proc. of the 2014 IEEE International Symposium on Workload Characterization (IISWC),* pp. 221-231, 2014.

[16] A. Reuel et al., “BetterBench: Assessing AI Benchmarks, Uncovering Issues, and Establishing Best Practices,” *Proc. of the 37th Advances in Neural Information Processing Systems (NeurIPS),* 2024.

[17] V. J. Reddi et al., “MLPerf Inference Benchmark,” *Proc. of the 2020 ACM/IEEE 47th Annual International Symposium on Computer Architecture (ISCA)*, pp. 446-459, 2020.

[18] Luo Chunjie, et al., “AIoT Bench: Towards Comprehensive Benchmarking Mobile and Embedded Device Intelligence,” *Proc. of the International Symposium on Benchmarking, Measuring and Optimization*, pp. 31-35, 2018.

[19] A. Ignatov et al., “AI Benchmark: All About Deep Learning on Smartphones in 2019,” *Proc. of the*

IEEE/CVF International Conference on Computer Vision Workshops (ICCVW), pp. 3617-3635. 2019.
[20] S. P. Baller, A. Jindal, M. Chadha and M. Gerndt, “DeepEdgeBench: Benchmarking Deep Neural Networks on Edge Devices,” *Proc. of the 2021 IEEE International Conference on Cloud Engineering,* pp. 20-30, 2021.
[21] Hasanpour, M. A., Kirkegaard, M., & Fafoutis, X. “EdgeMark: An Automation and Benchmarking System for Embedded Artificial Intelligence Tools,” *arXiv preprint,* arXiv:2502.01700, Feb. 2025.
[22] He, Kaiming, et al., “Deep Residual Learning for Image Recognition,” *Proc. of the 2016 IEEE Conference on Computer Vision and Pattern Recognition*, pp. 770-778, 2016.
[23] Howard, Andrew G., et al., “MobileNets: Efficient Convolutional Neural Networks for Mobile Vision Applications,” *arXiv preprint* arXiv:1704.04861, Apr. 2017.
[24] Raspberry Pi Foundation. Raspberry Pi 5 Tech specs [Online]. Available: https://www.raspberrypi.com/products/raspberry-pi-5. (downloaded Nov. 15, 2025).
[25] Raspberry Pi Foundation. Raspberry Pi 4 Tech specs [Online]. Available: https://www.raspberrypi.com/products/raspberry-pi-4-model-b/specifications. (downloaded Nov. 15, 2025).
[26] Raspberry Pi Foundation. Raspberry Pi 3B+ Tech specs [Online]. Available: https://www.raspberrypi.com/products/raspberry-pi-3-model-b-plus/. (downloaded Nov. 15, 2025).
[27] Texas Instruments. INA219 Precision Power Monitor [Online]. Available: https://www.ti.com/product/INA219 (downloaded Nov. 15, 2025).

권 용 훈

2019년 강원대학교 원예학과 졸업(학사) 2021년 강원대학교 컴퓨터정보통신공학과 졸업(석사). 2026년 강원대학교 컴퓨터정보통신공학과 졸업(박사). 관심분야는 사물인터넷, 엣지컴퓨팅, 임베디드시스템, 인공지능

정 인 범

1985년 고려대학교 전자공학과 졸업(학사) 1985년~1995년 ㈜삼성전자 컴퓨터 시스템 사업부 선임 연구원. 1994년 한국과학기술원 정보통신공학과 졸업(석사). 2000년 한국과학기술원 전산학과 졸업(박사). 2001년~현재 강원대학교 컴퓨터공학과 교수. 관심분야는 운영체제, 소프트웨어공학, 엣지컴퓨팅, 사물인터넷